

# FTXUI

## com CMake

FTXUI é uma biblioteca C++ para construção de UIs baseadas puramente em terminal, permitindo que você construa uma interface como uma árvore de elementos (Widgets) que se ajustam automaticamente.

## Iniciando

### Instalação

Para instalar a biblioteca FTXUI, você pode usar o CMake para compilar e instalar os arquivos necessários. Siga os passos abaixo:

1. No seu projeto, crie um arquivo `CMakeLists.txt` com o seguinte conteúdo:

```
cmake_minimum_required(VERSION 3.14)
project(ftxui VERSION 0.1.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# --- DEPENDENCIES (FTXUI) ---
include(FetchContent)
FetchContent_Declare(ftxui
  GIT_REPOSITORY [https://github.com/ArthurSonzogni/FTXUI](https://github.com/ArthurSonzogni/FTXUI)
  GIT_TAG v5.0.0
)
FetchContent_MakeAvailable(ftxui)

# --- APP EXECUTABLE ---
add_executable(ftxui_app src/main.cpp)

target_link_libraries(ftxui_app PRIVATE
  ftxui::screen
  ftxui::dom
  ftxui::component
)
```

2. Certifique-se de que o arquivo `main.cpp` esteja localizado na pasta `src` do seu projeto.

3. No terminal, navegue até o diretório do seu projeto e execute os seguintes comandos:

```
mkdir build
cd build
cmake ..
make
```

1. Após a compilação, você encontrará o executável `ftxui_app` na pasta `build`. Execute-o para ver sua aplicação em ação:

```
./ftxui_app
```

## Exemplo Simples

Aqui está um exemplo simples de como criar uma aplicação FTXUI que exibe um texto na tela:

```
#include "ftxui/component/component.hpp"
#include "ftxui/component/screen_interactive.hpp"
#include "ftxui/dom/elements.hpp"

using namespace ftxui;

int main() {
    auto screen = ScreenInteractive::TerminalOutput();

    auto renderer = Renderer([] {
        return text("Olá, FTXUI!") | center | border;
    });

    screen.Loop(renderer);
    return 0;
}
```

Este código cria uma aplicação que exibe o texto “Olá, FTXUI!” centralizado na tela com uma borda ao redor.

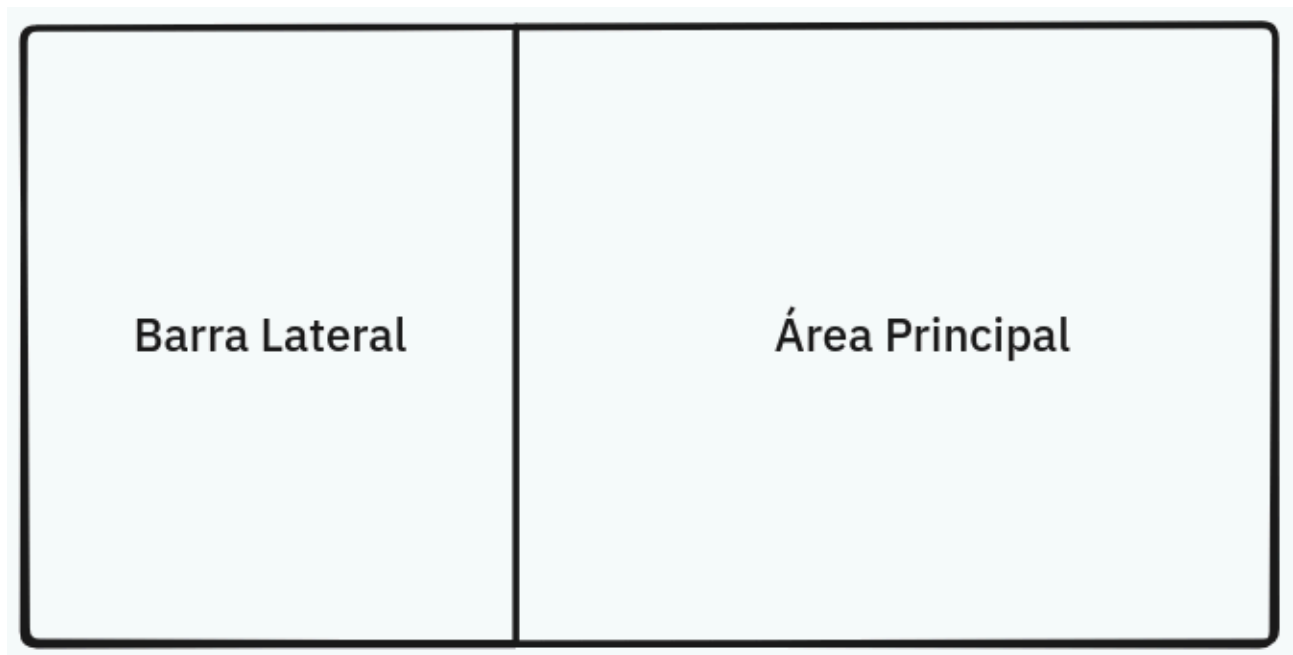
# Elementos Principais

## Contentores

São elementos que podem conter outros elementos, permitindo a criação de layouts complexos. Os principais contentores em FTXUI são:

- **hbox** (Horizontal Box) : Coloca os elementos lado a lado.
- **vbox** (Vertical Box) : Coloca os elementos um sobre o outro.

Igual ao HTML com `div`s e `span`s, você pode usar **hbox** e **vbox** para estruturar sua interface. Por exemplo, para criar uma estrutura como:



Precisamos, inicialmente, de um container “pai”, um **hbox**, que conterá as duas colunas como filhos. Cada coluna será um **vbox** que, por sua vez, terão seus próprios filhos. Por exemplo:

```
Element document = hbox({
    vbox({
        text("Barra Lateral"),
    }) | border,
    vbox({
        text("Área Principal")
    }) | border | flex,
});
```

O operador **| border** adiciona uma borda ao redor do elemento, enquanto o operador **| flex** faz com que o elemento expanda para preencher o espaço disponível. Porém, temos um único problema que é, desta forma ambas as colunas terão o tamanho dependente do tamanho do conteúdo da “Barra Lateral”, pois ela preencherá apenas o espaço que precisar enquanto a “Área Principal” toma todo o resto. Para resolver isso, precisamos definir dimensões fixas para ambos, ou apenas para os não flexíveis. Podemos fazer isso

usando o operador | **size(WIDTH, HEIGHT)**. Por exemplo, para definir a largura da “Barra Lateral” para 30% da largura total da tela, podemos fazer:

```
vbox({
    text("Barra Lateral"),
}) | border | size(WIDTH, EQUAL, 30)
```

*Outros contentores:*

- *zbox* : Empilha elementos em “camadas” (eixo Z), um em cima do outro.
- *gridbox* : Cria uma malha/tabela alinhada.

## Texto e Conteúdo

- **text(“”)** : A unidade básica. Escreve uma string.
- **paragraph(“texto longo...”)** : Igual ao text, mas faz “wrap” automático (quebra de linha) se não couber. Essencial para descrições de tarefas.
- **separator()** : Desenha uma linha (horizontal ou vertical dependendo do contexto). Ótimo para separar itens da lista ou o cabeçalho do conteúdo.
- **gauge(0.5)** : Uma barra de progresso (0.0 a 1.0). Perfeito para indicar a conclusão de um projeto.
- **checkbox(“label”, &bool\_variable)** : Uma caixa de seleção vinculada a uma variável booleana.
- **radiobox({“opção1”, “opção2”}, &selected\_index)** : Um grupo de botões de opção vinculados a um índice selecionado.
- **input(“placeholder”, &string\_variable)** : Um campo de entrada de texto vinculado a uma variável string.

## Decoradores (Operadores)

Estes modificam um elemento existente. Usas com o pipe |.

Bordas e Molduras:

- | **border** : A borda simples que já usaste.
- | **borderDouble** : Borda dupla (estilo mais “retro” ou para destacar a janela ativa).
- | **borderRounded** : Borda com cantos arredondados.
- | **window(“Título”, ...)** : Um atalho que cria uma borda com um título em cima.

**Cores e Estilo:**

- | **bold** (negrito), | **dim** (escurecido), | **inverted** (inverte fundo/frente).
- | **color(Color::Red)** : Muda a cor do texto.
- | **bgcolor(Color::Blue)** : Muda a cor do fundo.

Cores úteis: Red, Green, Blue, Yellow, Cyan, Magenta, White, Black.

## Tamanho e Layout :

- | **flex** : Ocupa o espaço que sobrar. Fundamental.
- | **center** : Centraliza o conteúdo dentro do espaço disponível.
- | **align\_right**, | **align\_left** : Alinhamento.
- | **size(WIDTH, EQUAL, 10)** : Força um tamanho. Pode usar WIDTH ou HEIGHT, e EQUAL, LESS\_THAN, GREATER\_THAN.

## Interatividade

Como uma aplicação interativa, FTXUI permite que você lide com eventos de teclado e mouse. Você pode criar componentes interativos como botões, caixas de seleção e campos de entrada de texto. No exemplo simples anterior usamos `ScreenInteractive` para criar uma tela interativa que responde a eventos do usuário. A variável `renderer` é um componente que renderiza o conteúdo na tela e pode ser atualizado em resposta a eventos. Você pode usar o loop da tela para manter a aplicação rodando e respondendo a eventos até que o usuário decida sair.

No FTXUI, a interatividade vem de componentes pré-prontos (como `Menu`, `Input`, `Button`) que nós “colamos” na tela. Por exemplo, para criar um menu interativo, você pode usar o componente `Menu`:

```
std::vector<std::string> entries = {
    "Todos os Projetos",
    "Em Andamento",
    "Concluídos",
    "Configurações",
    "Sair",
};
int selected = 0;

auto menu = Menu(&entries, &selected);
```

Neste exemplo, criamos um menu com várias opções e vinculamos a variável `selected` para rastrear qual opção está atualmente selecionada. Você pode então renderizar este menu na tela adicionando-o com `componente->Render()`.

```
auto renderer = Renderer([&]() {
    return menu->Render() | border | size(WIDTH, EQUAL, 30);
});
```

Porém, neste estado atual o menu não faz nada quando o usuário interage com ele. Para adicionar funcionalidade, você pode usar o loop da tela para processar eventos e atualizar a interface conforme necessário.

```
auto menu = Menu(&entries, &selected);

auto layout = Container::Horizontal({
    menu
});
```

```

auto renderer = Renderer(layout, [&]() {
    return hbox({
        menu->Render() | border | size(WIDTH, EQUAL, 30),
        vbox({
            text("Opção selecionada: " + entries[selected])
        }) | border | flex,
    });
});

```

Neste exemplo, criamos um container responsável por enviar eventos para componentes, neste caso um layout horizontal, que contém o menu que mostra a opção atualmente selecionada. O loop da tela mantém a aplicação rodando e respondendo a eventos do usuário, atualizando a exibição conforme o usuário navega pelo menu.

Agora que conseguimos lidar com inputs do usuário, podemos adicionar ações específicas para cada opção do menu. Por exemplo, quando o usuário seleciona “Sair”, podemos encerrar a aplicação:

```

MenuOption options;
options.on_enter = [&] {
    if (entries[selected] == "Sair") {
        screen.ExitLoopClosure()();
    }
};

auto menu = Menu(&entries, &selected, options);

```

Aqui criamos um objeto MenuOption onde definimos uma função lambda para o evento on\_enter, que é chamado quando o usuário pressiona Enter em uma opção do menu. Se a opção selecionada for “Sair”, chamamos screen.ExitLoopClosure()() para encerrar o loop da aplicação e sair. Este objeto é, então, passado para o construtor do Menu para associar o comportamento ao menu interativo.

*Obs: o (entries[selected] == “Sair”) também pode ser substituído por (selected == 4), já que “Sair” é a quinta opção (índice 4) na lista.*

## Componentes interativos

- Menu(std::vector, index, MenuOption): Cria um menu interativo com várias opções.
- Button(“Texto do Botão”, funcao\_callback): Quando ativado, ele executa a função lambda que você passou.
- Input(&string\_variavel, “placeholder”): você passa o endereço de uma std::string. Tudo o que o usuário digitar vai magicamente para essa string em tempo real.
- Checkbox(“Texto da Label”, &bool\_variavel): Liga-se a um bool. Se for true, aparece marcado.
- Radiobox(&lista\_opcoes, &int\_selecionado): Liga-se a um índice inteiro que indica qual opção está selecionada.
- Toggle(&lista\_opcoes, &int\_selecionado): Muito parecido com o Radiobox, mas visualmente parece uma “aba” ou botões colados horizontalmente.

- Slider("Label", &int\_variavel, min, max): Uma barra deslizante para escolher números. Definir % de progresso manual ou complexidade da tarefa.

## Abas (Tabs)

FTXUI também suporta a criação de interfaces com abas (tabs), permitindo que você organize o conteúdo em diferentes seções que podem ser acessadas através de uma barra de abas. Para criar abas em FTXUI, você pode usar o componente Tab:

```
auto tab_projetos = Renderer([] {
    return vbox({ text("Aba 1") });
});
auto tab_config = Renderer([] {
    return text("Aba 2");
});

// Container::Tab usa o mesmo índice 'selected' do menu!
auto tab_container = Container::Tab({
    tab_projetos,
    tab_config,
}, &selected);

auto layout = Container::Horizontal({
    menu,
    tab_container | flex,
});
```

## Manipulação de Dados e Estado

Para criar listas de tarefas reais que crescem e diminuem, precisamos usar estruturas de dados dinâmicas.

### O Problema do std::vector

Muitos componentes do FTXUI (como Checkbox) armazenam um ponteiro para a variável booleana que eles controlam (&tarefa.concluida). Se usarmos std::vector, ao adicionar novos itens, o vetor pode realocar memória, invalidando esses ponteiros e causando falhas (crash). **Solução:** Use std::list ou std::deque, que garantem estabilidade de ponteiros.

```
#include <list>
struct Tarefa {
    std::string titulo;
    bool concluida;
};
std::list<Tarefa> lista_tarefas;
```

### Renderização Reativa

Podemos gerar elementos visuais dinamicamente dentro do Renderer iterando sobre nossa lista de dados. Isso garante que a UI esteja sempre sincronizada com os dados.

```

auto renderer = Renderer([&] {
    Elements linhas;
    for (auto& tarefa : lista_tarefas) {
        linhas.push_back(hbox({
            text(tarefa.concluida ? "[x] " : "[ ] "),
            text(tarefa.titulo)
        }));
    }
    return vbox(linhas);
});

```

## Eventos e Controle Avançado

Para criar teclas de atalho globais ou manipular o foco manualmente, usamos o sistema de eventos.

### CatchEvent

O decorador CatchEvent permite interceptar teclas antes que elas cheguem a um componente.

```

auto componente_com_atalho = componente | CatchEvent([&](Event event) {
    // Detectar tecla 'Ctrl + x'
    if (event == Event::Ctrl('x')) {
        screen.ExitLoopClosure(); // Fecha o programa
        return true; // Evento consumido
    }

    // Detectar tecla 'Delete'
    if (event == Event::Delete) {
        // Lógica para deletar item...
        return true;
    }

    return false; // Passa o evento adiante (não consumido)
});

```

### Gerenciamento de Foco (Depth)

Em layouts complexos (Menu à esquerda, Lista à direita), precisamos controlar onde está o foco do teclado. O Container aceita um ponteiro para um inteiro (selector ou depth) que define qual filho está ativo.

```

int depth = 0; // 0 = Menu, 1 = Abas

auto layout_principal = Container::Horizontal({
    menu, // Índice 0
    tab_container, // Índice 1
}, &depth);

```

Podemos então manipular essa variável depth programaticamente:

- Ao apertar Enter no Menu -> depth = 1 (Vai para a lista).
- Ao apertar Esc na Lista -> depth = 0 (Volta para o menu).